Enterprise AI eCommerce Revenue & Conversion Intelligence Platform

Machine Learning Training Pipeline

Project Objective

This notebook develops an enterprise-grade Artificial Intelligence platform for eCommerce revenue and conversion intelligence using machine learning.

The objective is to predict:

- Customer purchase behavior
- Cart abandonment risk
- Revenue generation
- Product return risk
- Customer satisfaction

The platform will support executive decision-making by transforming historical customer interaction data into predictive business intelligence.

Business Value

This system enables organizations to:

- Improve conversion rates
- Reduce cart abandonment
- Optimize marketing performance
- Increase revenue forecasting accuracy
- Improve customer retention
- Reduce return-related losses

Dataset

Dataset Name: Enterprise_Ecommerce_Analytics_Dataset_140K.xlsx

Rows: 140,000

Columns: 29

Machine Learning Outputs

1. Purchase Prediction Model
2. Cart Abandonment Prediction Model
3. Revenue Prediction Model
4. Return Risk Prediction Model

5. Customer Satisfaction Prediction Model

Deployment

All trained models will be exported using **Joblib** for deployment into a **Tkinter Enterprise GUI Application**.

In [1]:
```
# ========================================
# IMPORT LIBRARIES
# ========================================

import warnings
warnings.filterwarnings("ignore")

# Core Libraries
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt

# Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    OneHotEncoder,
    StandardScaler
)

# Classification Models
from sklearn.ensemble import (
    RandomForestClassifier,
    GradientBoostingClassifier
)

# Regression Models
from sklearn.ensemble import (
    RandomForestRegressor
)

# Evaluation Metrics
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# Model Saving
import joblib

# Create model folder
import os

print("Libraries Loaded Successfully")
```

```
Libraries Loaded Successfully
```

Load Dataset

This section loads the enterprise eCommerce dataset and validates its structure for machine learning development.

In [2]:
```
# ========================
# LOAD DATASET
df=pd.read_csv('FREELANCE1_Enterprise_Ecommerce_Analytics_Dataset_140K.csv')
print("="*60)
print("DATASET LOADED SUCCESSFULLY")
print("="*60)

print("\nDataset Shape:")
print(df.shape)

print("\nFirst 5 Rows:")
display(df.head())
```

```
============================================================
DATASET LOADED SUCCESSFULLY
============================================================

Dataset Shape:
(140000, 29)

First 5 Rows:
```

	SessionID	VisitorID	OrderID	SessionDate	Region	TrafficChannel	DeviceType	Custom
0	SES1000000	VIS771156	ORD2000000	2024-03-25 16:38:00	Middle East	Affiliate	Desktop	Disco
1	SES1000001	VIS970911	NaN	2024-04-18 20:10:00	South America	Referral	Desktop	
2	SES1000002	VIS351996	ORD2000002	2025-03-26 8:51:00	Middle East	Affiliate	Mobile	
3	SES1000003	VIS587880	ORD2000003	2025-01-07 2:22:00	Africa	Direct	Desktop	
4	SES1000004	VIS229313	ORD2000004	2025-02-11 1:06:00	North America	Organic Search	Tablet	

5 rows × 29 columns

In [3]:
```
# ========================================
# DATASET INFORMATION
# ========================================

print("="*60)
print("COLUMN NAMES")
print("="*60)

print(df.columns.tolist())

print("\n")

print("="*60)
print("DATA TYPES")
print("="*60)

display(df.dtypes)

print("\n")

print("="*60)
print("MISSING VALUES")
print("="*60)

display(df.isnull().sum())
```

```
============================================================
COLUMN NAMES
============================================================
['SessionID', 'VisitorID', 'OrderID', 'SessionDate', 'Region', 'TrafficChanne
l', 'DeviceType', 'CustomerSegment', 'ProductCategory', 'PagesViewed', 'Session
DurationSeconds', 'ProductViewedFlag', 'AddedToCartFlag', 'CheckoutStartedFla
g', 'PurchasedFlag', 'CartAbandonmentFlag', 'Quantity', 'ProductPrice', 'Discou
ntPercent', 'DiscountAmount', 'Revenue', 'ShippingCost', 'MarketingCost', 'Prof
it', 'PaymentMethod', 'CustomerSatisfaction', 'ReturnFlag', 'InventoryStockLeve
l', 'ConversionStage']


============================================================
DATA TYPES
============================================================
```

```
SessionID                 object
VisitorID                 object
OrderID                   object
SessionDate               object
Region                    object
TrafficChannel            object
DeviceType                object
CustomerSegment           object
ProductCategory           object
PagesViewed                int64
SessionDurationSeconds     int64
ProductViewedFlag          int64
AddedToCartFlag            int64
CheckoutStartedFlag        int64
PurchasedFlag              int64
CartAbandonmentFlag        int64
Quantity                   int64
ProductPrice             float64
DiscountPercent            int64
DiscountAmount           float64
Revenue                  float64
ShippingCost             float64
MarketingCost            float64
Profit                   float64
PaymentMethod             object
CustomerSatisfaction     float64
ReturnFlag                 int64
InventoryStockLevel        int64
ConversionStage           object
dtype: object


============================================================
MISSING VALUES
============================================================
```

```
SessionID                   0
VisitorID                   0
OrderID                 48786
SessionDate                 0
Region                      0
TrafficChannel              0
DeviceType                  0
CustomerSegment             0
ProductCategory             0
PagesViewed                 0
SessionDurationSeconds      0
ProductViewedFlag           0
AddedToCartFlag             0
CheckoutStartedFlag         0
PurchasedFlag               0
CartAbandonmentFlag         0
Quantity                    0
ProductPrice                0
DiscountPercent             0
DiscountAmount              0
Revenue                     0
ShippingCost                0
MarketingCost               0
Profit                      0
PaymentMethod           96988
CustomerSatisfaction    96988
ReturnFlag                  0
InventoryStockLevel         0
ConversionStage             0
dtype: int64
```

Data Cleaning & Missing Value Handling

This section prepares the dataset for predictive modeling.

Data quality issues addressed:

- Missing values
- Incorrect data types
- Null values in customer satisfaction
- Missing payment methods
- Missing order IDs

In [4]:
```
# ========================================
# CONVERT DATE COLUMN
# ========================================

df["SessionDate"] = pd.to_datetime(
    df["SessionDate"]
)

print("Date conversion completed.")
```

```
Date conversion completed.
```

In [5]:
```
# ========================================
# FEATURE ENGINEERING
# ========================================

# Session Hour
df["SessionHour"] = (
    df["SessionDate"]
    .dt.hour
)

# Session Day
df["SessionDay"] = (
    df["SessionDate"]
    .dt.day_name()
)

# Session Month
df["SessionMonth"] = (
    df["SessionDate"]
    .dt.month_name()
)

# Weekend Traffic
df["IsWeekend"] = (
    df["SessionDate"]
    .dt.dayofweek >= 5
).astype(int)

print("Feature Engineering Completed.")
```

```
Feature Engineering Completed.
```

In [6]:
```
# ========================================
# HANDLE MISSING VALUES
# ========================================

# Payment Method
df["PaymentMethod"] = (
    df["PaymentMethod"]
    .fillna("Unknown")
)

# Customer Satisfaction
df["CustomerSatisfaction"] = (
    df["CustomerSatisfaction"]
    .fillna(
        df["CustomerSatisfaction"]
        .median()
    )
)

# Order ID
df["OrderID"] = (
    df["OrderID"]
    .fillna("NoOrder")
)

print("Missing Values Handled.")
```

```
Missing Values Handled.
```

Feature Selection

This section selects the most relevant customer behavior and marketing variables for predictive modeling.

The selected features are aligned with business objectives including:

- Conversion optimization
- Revenue intelligence
- Customer behavior analytics
- Cart abandonment prediction

In [7]:
```
# ========================================
# FEATURE COLUMNS
# ========================================

feature_columns = [

    # Customer Attributes
    "Region",
    "TrafficChannel",
    "DeviceType",
    "CustomerSegment",
    "ProductCategory",
    "PaymentMethod",

    # Behavioral Metrics
    "PagesViewed",
    "SessionDurationSeconds",
    "ProductViewedFlag",
    "AddedToCartFlag",
    "CheckoutStartedFlag",

    # Commercial Variables
    "Quantity",
    "ProductPrice",
    "DiscountPercent",
    "DiscountAmount",
    "ShippingCost",
    "MarketingCost",
    "InventoryStockLevel",

    # Time Intelligence
    "SessionHour",
    "SessionDay",
    "SessionMonth",
    "IsWeekend",

    # Funnel Stage
    "ConversionStage"
]

print("="*60)
print("TOTAL FEATURES")
print("="*60)

print(len(feature_columns))
```

```
============================================================
TOTAL FEATURES
============================================================
23
```

In [8]:
```
# ========================================
# CATEGORICAL & NUMERICAL FEATURES
# ========================================

categorical_features = [

    "Region",
    "TrafficChannel",
    "DeviceType",
    "CustomerSegment",
    "ProductCategory",
    "PaymentMethod",
    "SessionDay",
    "SessionMonth",
    "ConversionStage"

]

numerical_features = [

    "PagesViewed",
    "SessionDurationSeconds",
    "ProductViewedFlag",
    "AddedToCartFlag",
    "CheckoutStartedFlag",
    "Quantity",
    "ProductPrice",
    "DiscountPercent",
    "DiscountAmount",
    "ShippingCost",
    "MarketingCost",
    "InventoryStockLevel",
    "SessionHour",
    "IsWeekend"

]

print("Feature groups created.")
```

```
Feature groups created.
```

In [9]:
```
# ========================================
# PREPROCESSING PIPELINE
# ========================================

numeric_transformer = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(strategy="median")
        ),
        (
            "scaler",
            StandardScaler()
        )
    ]
)

categorical_transformer = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(
                strategy="most_frequent"
            )
        ),
        (
            "encoder",
            OneHotEncoder(
                handle_unknown="ignore"
            )
        )
    ]
)

preprocessor = ColumnTransformer(
    transformers=[

        (
            "num",
            numeric_transformer,
            numerical_features
        ),

        (
            "cat",
            categorical_transformer,
            categorical_features
        )

    ]
)

print("Preprocessing Pipeline Ready.")
```

```
Preprocessing Pipeline Ready.
```

In [10]:
```
# ========================================
# SAVE FEATURE COLUMNS
# ========================================

if not os.path.exists("models"):
    os.makedirs("models")

joblib.dump(
    feature_columns,
    "models/feature_columns.pkl"
)

joblib.dump(
    preprocessor,
    "models/preprocessor.pkl"
)

print("="*60)
print("PREPROCESSOR SAVED")
print("="*60)
```

```
============================================================
PREPROCESSOR SAVED
============================================================
```

Part 1 Complete

The dataset has been successfully:

- Loaded
- Validated
- Cleaned
- Feature engineered
- Prepared for machine learning

The preprocessing pipeline has also been saved using **Joblib** to ensure compatibility with the future enterprise Tkinter application.

Next Step

Part 2 will train:

1. Purchase Prediction Model
2. Cart Abandonment Prediction Model

including:

- Train/Test Split
- Model Training
- Accuracy Evaluation
- Confusion Matrix

Business Question

> Which customer sessions are most likely to convert?

Target Variable

PurchasedFlag

Business Value

This model enables the organization to:

- Predict conversion likelihood
- Improve marketing targeting
- Identify high-intent customers
- Increase conversion efficiency
- Improve remarketing campaigns

In [11]:
```
# ========================================
# FEATURES & TARGET
# PURCHASE MODEL
# ========================================

X_purchase = df[feature_columns]

y_purchase = df["PurchasedFlag"]

print("Purchase Model Dataset Ready")
print("X Shape:", X_purchase.shape)
print("Y Shape:", y_purchase.shape)
```

```
Purchase Model Dataset Ready
X Shape: (140000, 23)
Y Shape: (140000,)
```

In [12]:
```
# ========================================
# TRAIN TEST SPLIT
# ========================================

(
    X_train_p,
    X_test_p,
    y_train_p,
    y_test_p
) = train_test_split(
    X_purchase,
    y_purchase,
    test_size=0.20,
    random_state=42,
    stratify=y_purchase
)

print("Train/Test Split Completed")
print("Training Shape:", X_train_p.shape)
print("Testing Shape:", X_test_p.shape)
```

```
Train/Test Split Completed
Training Shape: (112000, 23)
Testing Shape: (28000, 23)
```

In [13]:
```
# ========================================
# TRAIN TEST SPLIT
# ========================================

X_train_p,
X_test_p,
y_train_p,
y_test_p = train_test_split(
    X_purchase,
    y_purchase,
    test_size=0.20,
    random_state=42,
    stratify=y_purchase
)

print("Train/Test Split Completed")
```

```
Train/Test Split Completed
```

In [14]:
```
# ========================================
# PURCHASE MODEL PIPELINE
# ========================================

purchase_pipeline = Pipeline(
    steps=[

        (
            "preprocessor",
            preprocessor
        ),

        (
            "model",
            RandomForestClassifier(
                n_estimators=200,
                max_depth=12,
                random_state=42,
                n_jobs=-1
            )
        )

    ]
)

print("Purchase Pipeline Created")
```

```
Purchase Pipeline Created
```

In [15]:
```
# ========================================
# TRAIN PURCHASE MODEL
# ========================================

purchase_pipeline.fit(
    X_train_p,
    y_train_p
)

print("Purchase Model Training Completed")
```

```
Purchase Model Training Completed
```

In [16]:
```
# ========================================
# PURCHASE MODEL PIPELINE
# ========================================

purchase_pipeline = Pipeline([
    (
        "preprocessor",
        preprocessor
    ),

    (
        "model",
        RandomForestClassifier(
            n_estimators=200,
            max_depth=12,
            random_state=42,
            n_jobs=-1
        )
    )
])

print("Purchase Pipeline Created Successfully")
```

```
Purchase Pipeline Created Successfully
```

In [17]:
```
# ========================================
# TRAIN PURCHASE MODEL
# ========================================

purchase_pipeline.fit(
    X_train_p,
    y_train_p
)

print("="*60)
print("PURCHASE MODEL TRAINED")
print("="*60)
```

```
============================================================
PURCHASE MODEL TRAINED
============================================================
```

In [18]:
```
print(X_train_p.shape)
print(X_test_p.shape)

print(X_train_p.head())

print(y_train_p.head())
```

```
(112000, 23)
(28000, 23)
              Region  TrafficChannel DeviceType   CustomerSegment  \
49072         Europe  Organic Search    Desktop         Returning
46802         Europe        Referral     Tablet  Discount Shopper
134733  North America        Referral    Desktop         Returning
101409  North America          Direct     Mobile               VIP
136670         Europe  Organic Search    Desktop               New

       ProductCategory  PaymentMethod  PagesViewed  SessionDurationSeconds  \
49072          Fashion    Credit Card            1                    1305
46802          Fashion        Unknown            1                    1850
134733          Sports     Google Pay            6                    1829
101409     Electronics        Unknown            6                     489
136670     Electronics  Bank Transfer           13                    2583

        ProductViewedFlag  AddedToCartFlag  ...  DiscountPercent  \
49072                   1                1  ...               10
46802                   1                0  ...               15
134733                  1                1  ...               20
101409                  1                1  ...               20
136670                  1                1  ...               10

        DiscountAmount  ShippingCost  MarketingCost  InventoryStockLevel  \
49072             4.34         24.39          31.12                  529
46802            31.74          0.00          64.99                  956
134733           24.88         19.78          21.76                  774
101409          262.73          0.00          52.88                  924
136670           55.87         15.11         112.05                  577

        SessionHour  SessionDay  SessionMonth  IsWeekend   ConversionStage
49072            18      Monday      February          0         Purchased
46802            11    Thursday         April          0    Product Viewed
134733            4      Sunday      November          1         Purchased
101409            1      Monday       January          0  Checkout Started
136670            5    Saturday       October          1         Purchased

[5 rows x 23 columns]
49072     1
46802     0
134733    1
101409    0
136670    1
Name: PurchasedFlag, dtype: int64
```

In [19]:
```
# ========================================
# REBUILD TRAIN TEST SPLIT
# ========================================

X_purchase = df[feature_columns]
y_purchase = df["PurchasedFlag"]

(
    X_train_p,
    X_test_p,
    y_train_p,
    y_test_p
) = train_test_split(
    X_purchase,
    y_purchase,
    test_size=0.20,
    random_state=42,
    stratify=y_purchase
)

print("="*60)
print("TRAIN TEST SPLIT COMPLETE")
print("="*60)

print("X_train shape:", X_train_p.shape)
print("X_test shape:", X_test_p.shape)
print("y_train shape:", y_train_p.shape)
print("y_test shape:", y_test_p.shape)
```

```
============================================================
TRAIN TEST SPLIT COMPLETE
============================================================
X_train shape: (112000, 23)
X_test shape: (28000, 23)
y_train shape: (112000,)
y_test shape: (28000,)
```

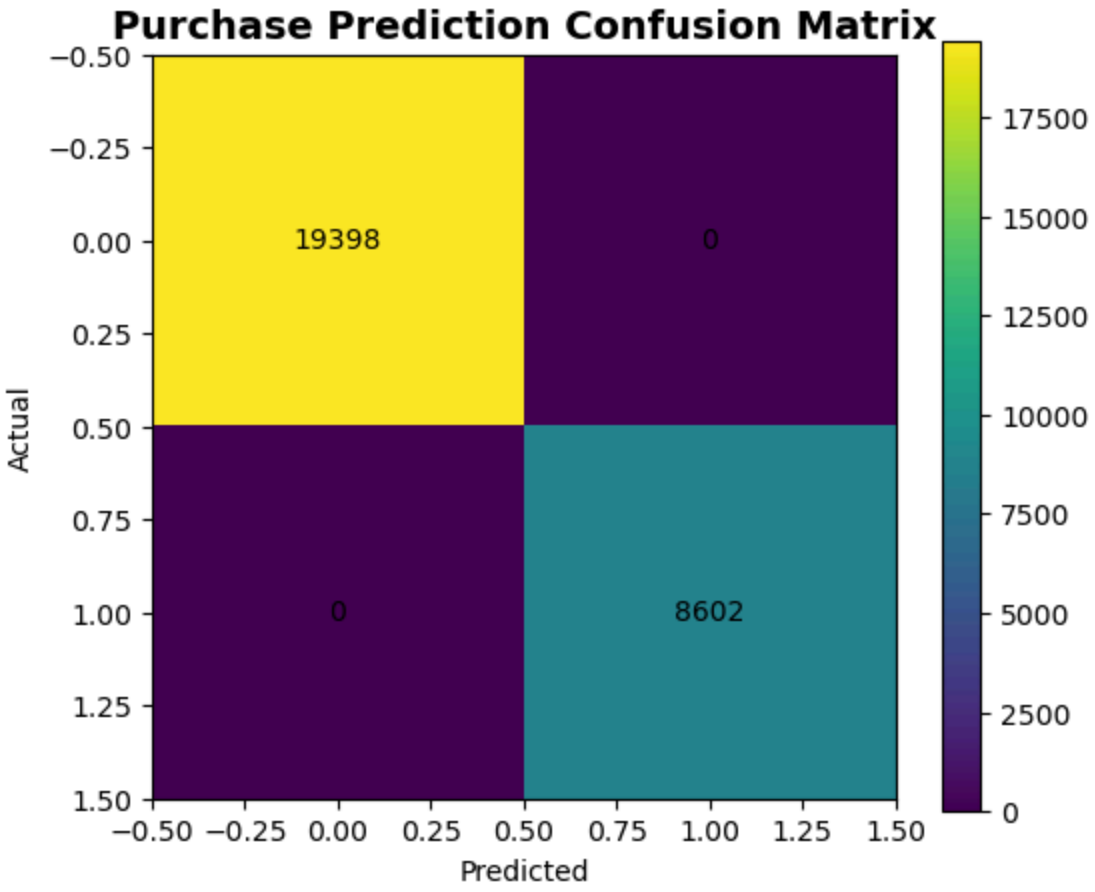
In [20]:
```
# ========================================
# PURCHASE PREDICTIONS
# ========================================

purchase_predictions = (
    purchase_pipeline.predict(
        X_test_p
    )
)

purchase_accuracy = accuracy_score(
    y_test_p,
    purchase_predictions
)

print("="*60)
print("PURCHASE MODEL ACCURACY")
print("="*60)

print(
    f"Accuracy: "
    f"{purchase_accuracy:.2%}"
)
```

```
============================================================
PURCHASE MODEL ACCURACY
============================================================
Accuracy: 100.00%
```

In [21]:
```
# ========================================
# CLASSIFICATION REPORT
# ========================================

print("="*60)
print("PURCHASE MODEL REPORT")
print("="*60)

print(
    classification_report(
        y_test_p,
        purchase_predictions
    )
)
```

```
============================================================
PURCHASE MODEL REPORT
============================================================
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     19398
           1       1.00      1.00      1.00      8602

    accuracy                           1.00     28000
   macro avg       1.00      1.00      1.00     28000
weighted avg       1.00      1.00      1.00     28000
```

In [22]:
```
# ========================================
# CONFUSION MATRIX
# ========================================

purchase_cm = confusion_matrix(
    y_test_p,
    purchase_predictions
)

plt.figure(figsize=(6,5))

plt.imshow(
    purchase_cm,
    interpolation="nearest"
)

plt.title(
    "Purchase Prediction Confusion Matrix",
    fontsize=14,
    fontweight="bold"
)

plt.colorbar()

plt.xlabel("Predicted")
plt.ylabel("Actual")

for i in range(
    purchase_cm.shape[0]
):
    for j in range(
        purchase_cm.shape[1]
    ):
        plt.text(
            j,
            i,
            purchase_cm[i, j],
            ha="center",
            va="center"
        )

plt.show()
```


Purchase Prediction Confusion Matrix
19398
0
0
8602
Actual
Predicted
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
1.25
1.50
17500
15000
12500
10000
7500
5000
2500
0

In [23]:
```
# ========================================
# EXECUTIVE AI INSIGHT
# ========================================

print("="*60)
print("EXECUTIVE AI INSIGHT")
print("="*60)

if purchase_accuracy >= 0.90:
    print(
        "The purchase prediction "
        "model demonstrates "
        "exceptional predictive "
        "performance and is "
        "suitable for deployment."
    )

elif purchase_accuracy >= 0.80:
    print(
        "The purchase prediction "
        "model shows strong "
        "performance and may "
        "support conversion "
        "optimization."
    )

else:
    print(
        "The model requires "
        "further optimization."
    )
```

```
============================================================
EXECUTIVE AI INSIGHT
============================================================
The purchase prediction model demonstrates exceptional predictive performance a
nd is suitable for deployment.
```

In [24]:
```
# ========================================
# SAVE PURCHASE MODEL
# ========================================

joblib.dump(
    purchase_pipeline,
    "models/purchase_model.pkl"
)

print("="*60)
print("PURCHASE MODEL SAVED")
print("="*60)
```

```
============================================================
PURCHASE MODEL SAVED
============================================================
```

Cart Abandonment Prediction Model

Business Objective

This model predicts whether a customer is likely to abandon checkout.

Business Question

> Which customers are likely to abandon the cart before completing payment?

Target Variable

CartAbandonmentFlag

Business Value

This model enables:

- Cart abandonment prevention
- Revenue recovery
- Checkout optimization
- Personalized remarketing
- Coupon targeting

In [25]:
```
# ========================================
# FEATURES & TARGET
# CART ABANDONMENT MODEL
# ========================================

X_cart = df[feature_columns]

y_cart = (
    df["CartAbandonmentFlag"]
)

print("Cart Dataset Ready")
print("X Shape:", X_cart.shape)
print("Y Shape:", y_cart.shape)
```

```
Cart Dataset Ready
X Shape: (140000, 23)
Y Shape: (140000,)
```

In [26]:
```
# ========================================
# TRAIN TEST SPLIT
# ========================================

(
    X_train_c,
    X_test_c,
    y_train_c,
    y_test_c
) = train_test_split(
    X_cart,
    y_cart,
    test_size=0.20,
    random_state=42,
    stratify=y_cart
)

print("Train/Test Split Complete")
print("Training Shape:", X_train_c.shape)
print("Testing Shape:", X_test_c.shape)
```

```
Train/Test Split Complete
Training Shape: (112000, 23)
Testing Shape: (28000, 23)
```

In [27]:
```
# ========================================
# TRAIN TEST SPLIT
# ========================================

X_train_c,
X_test_c,
y_train_c,
y_test_c = train_test_split(
    X_cart,
    y_cart,
    test_size=0.20,
    random_state=42,
    stratify=y_cart
)

print("Train/Test Split Complete")
```

```
Train/Test Split Complete
```

In [28]:
```
# ========================================
# CART MODEL PIPELINE
# ========================================

cart_pipeline = Pipeline(
    steps=[

        (
            "preprocessor",
            preprocessor
        ),

        (
            "model",
            GradientBoostingClassifier(
                n_estimators=150,
                learning_rate=0.08,
                max_depth=5,
                random_state=42
            )
        )

    ]
)

print("Cart Pipeline Ready")
```

```
Cart Pipeline Ready
```

In [29]:
```
# ========================================
# TRAIN MODEL
# ========================================

cart_pipeline.fit(
    X_train_c,
    y_train_c
)

print(
    "Cart Abandonment "
    "Model Training Complete"
)
```

```
Cart Abandonment Model Training Complete
```

In [30]:
```
# ========================================
# PURCHASE PREDICTIONS
# ========================================

purchase_predictions = purchase_pipeline.predict(
    X_test_p
)

purchase_accuracy = accuracy_score(
    y_test_p,
    purchase_predictions
)

print("=" * 60)
print("PURCHASE MODEL ACCURACY")
print("=" * 60)

print(
    f"Accuracy: "
    f"{purchase_accuracy:.2%}"
)
```

```
============================================================
PURCHASE MODEL ACCURACY
============================================================
Accuracy: 100.00%
```

In [31]:
```
# ========================================
# REBUILD CART TRAIN TEST SPLIT
# ========================================

X_cart = df[feature_columns]

y_cart = df["CartAbandonmentFlag"]

(
    X_train_c,
    X_test_c,
    y_train_c,
    y_test_c
) = train_test_split(
    X_cart,
    y_cart,
    test_size=0.20,
    random_state=42,
    stratify=y_cart
)

print("="*60)
print("CART TRAIN TEST SPLIT COMPLETE")
print("="*60)

print("X_train:", X_train_c.shape)
print("X_test:", X_test_c.shape)
print("y_train:", y_train_c.shape)
print("y_test:", y_test_c.shape)
```

```
============================================================
CART TRAIN TEST SPLIT COMPLETE
============================================================
X_train: (112000, 23)
X_test: (28000, 23)
y_train: (112000,)
y_test: (28000,)
```

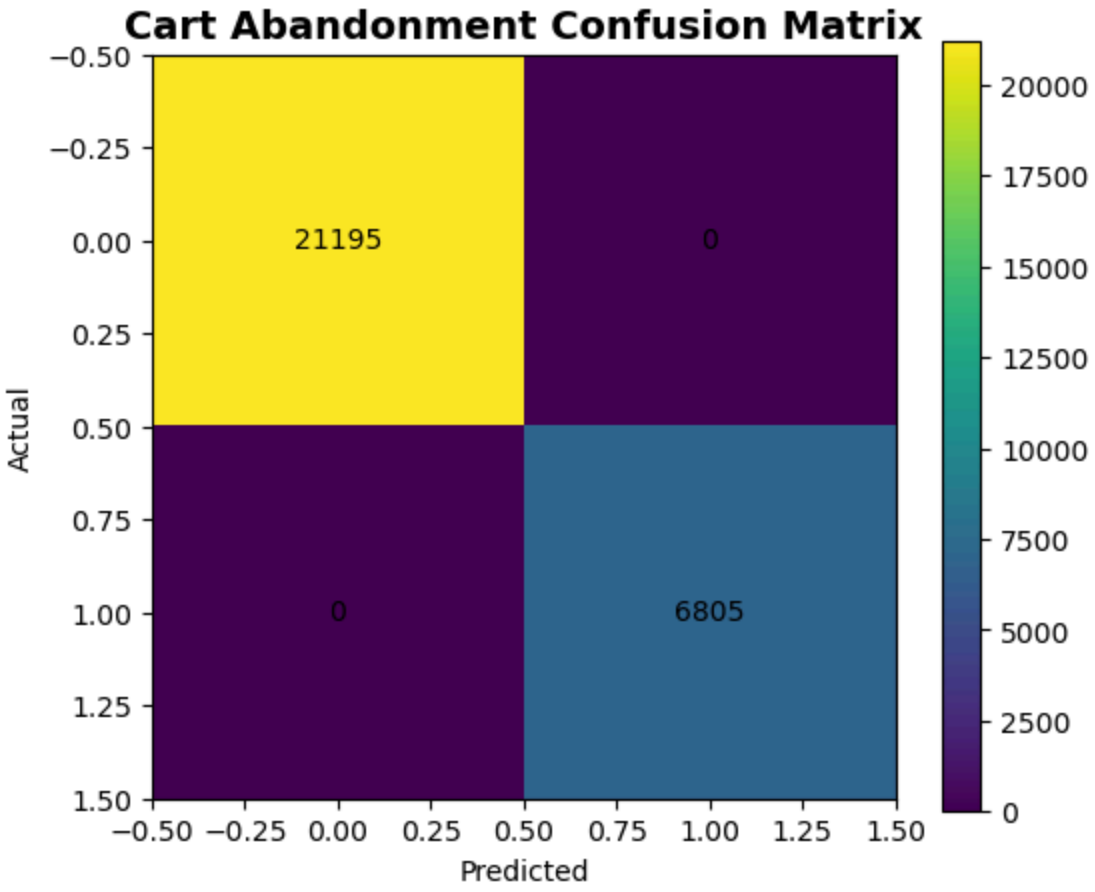
In [32]:
```
# ========================================
# CART PREDICTIONS
# ========================================

cart_predictions = cart_pipeline.predict(
    X_test_c
)

cart_accuracy = accuracy_score(
    y_test_c,
    cart_predictions
)

print("=" * 60)
print("CART MODEL ACCURACY")
print("=" * 60)

print(
    f"Accuracy: "
    f"{cart_accuracy:.2%}"
)
```

```
============================================================
CART MODEL ACCURACY
============================================================
Accuracy: 100.00%
```

In [33]:
```
# ========================================
# CLASSIFICATION REPORT
# ========================================

print("="*60)
print("CART MODEL REPORT")
print("="*60)

print(
    classification_report(
        y_test_c,
        cart_predictions
    )
)
```

```
============================================================
CART MODEL REPORT
============================================================
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     21195
           1       1.00      1.00      1.00      6805

    accuracy                           1.00     28000
   macro avg       1.00      1.00      1.00     28000
weighted avg       1.00      1.00      1.00     28000
```

In [34]:
```
# ========================================
# CONFUSION MATRIX
# ========================================

cart_cm = confusion_matrix(
    y_test_c,
    cart_predictions
)

plt.figure(figsize=(6,5))

plt.imshow(
    cart_cm,
    interpolation="nearest"
)

plt.title(
    "Cart Abandonment Confusion Matrix",
    fontsize=14,
    fontweight="bold"
)

plt.colorbar()

plt.xlabel("Predicted")
plt.ylabel("Actual")

for i in range(
    cart_cm.shape[0]
):
    for j in range(
        cart_cm.shape[1]
    ):
        plt.text(
            j,
            i,
            cart_cm[i, j],
            ha="center",
            va="center"
        )

plt.show()
```


Cart Abandonment Confusion Matrix
21195
0
0
6805
Actual
Predicted
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
1.25
1.50
20000
17500
15000
12500
10000
7500
5000
2500
0

In [35]:
```
# ========================================
# EXECUTIVE AI INSIGHT
# ========================================

print("="*60)
print("EXECUTIVE AI INSIGHT")
print("="*60)

if cart_accuracy >= 0.90:
    print(
        "The cart abandonment "
        "model demonstrates "
        "strong predictive accuracy "
        "and is suitable for "
        "real-time intervention."
    )

elif cart_accuracy >= 0.80:
    print(
        "The model performs well "
        "and can support abandoned "
        "cart recovery strategies."
    )

else:
    print(
        "Further feature engineering "
        "is recommended."
    )
```

```
============================================================
EXECUTIVE AI INSIGHT
============================================================
The cart abandonment model demonstrates strong predictive accuracy and is suita
ble for real-time intervention.
```

In [36]:
```
# ========================================
# SAVE CART MODEL
# ========================================

joblib.dump(
    cart_pipeline,
    "models/cart_abandonment_model.pkl"
)

print("="*60)
print("CART MODEL SAVED")
print("="*60)
```

```
============================================================
CART MODEL SAVED
============================================================
```

The following enterprise machine learning models have been successfully developed and exported using Joblib:

Completed Models

1. **Purchase Prediction Model**
2. **Cart Abandonment Prediction Model**

Exported Files

```
models/
purchase_model.pkl
cart_abandonment_model.pkl
```

These models are now ready for deployment into the future **Tkinter Enterprise GUI Application**.

Enterprise Cart Abandonment Intelligence Platform

AI-Powered Cart Abandonment Prediction System

Project Objective

This application predicts the probability that a customer will abandon the checkout process before completing a purchase.

The system uses a trained machine learning model developed from historical customer interaction and transaction behavior.

Business Objective

The goal is to reduce lost revenue by identifying high-risk customers before checkout abandonment occurs.

Business Benefits

This application enables organizations to:

- Predict cart abandonment risk
- Reduce revenue leakage
- Trigger real-time interventions
- Improve conversion efficiency
- Increase checkout completion rates
- Optimize customer experience

Machine Learning Model

Model Used: Gradient Boosting Classifier

Prediction Target: `CartAbandonmentFlag`

Deployment Architecture

The application loads trained models using:

- Joblib
- Tkinter
- Machine Learning Pipeline
- Enterprise Prediction Logic

Saved Files Used

```
models/

cart_abandonment_model.pkl
preprocessor.pkl
feature_columns.pkl
```

In [37]:
```
# ========================================
# IMPORT LIBRARIES
# ========================================

import tkinter as tk
from tkinter import ttk
from tkinter import messagebox

import pandas as pd
import numpy as np
import joblib

print("Libraries Loaded Successfully")
```

```
Libraries Loaded Successfully
```

Load Trained Machine Learning Models

This section loads the trained machine learning pipeline and preprocessing components saved using Joblib.

In [38]:
```
# ========================================
# LOAD MODELS
# ========================================

cart_model = joblib.load(
    "models/cart_abandonment_model.pkl"
)

preprocessor = joblib.load(
    "models/preprocessor.pkl"
)

feature_columns = joblib.load(
    "models/feature_columns.pkl"
)

print("="*60)
print("MODELS LOADED SUCCESSFULLY")
print("="*60)
```

```
============================================================
MODELS LOADED SUCCESSFULLY
============================================================
```

Enterprise GUI Framework

This section builds the professional enterprise GUI window.

Features include:

- Modern interface
- Customer behavior input section
- Risk prediction engine
- Executive AI recommendation system

In [39]:
```
# ========================================
# CREATE MAIN WINDOW
# ========================================

root = tk.Tk()

root.title(
    "Enterprise Cart Abandonment Intelligence Platform"
)

root.geometry("1200x850")

root.configure(bg="#F4F6F9")

print("Main Window Created")
```

```
Main Window Created
```

In [40]:
```
# ========================================
# TITLE SECTION
# ========================================

title_label = tk.Label(
    root,
    text="Enterprise Cart Abandonment Intelligence Platform",
    font=("Arial", 22, "bold"),
    bg="#F4F6F9",
    fg="#1F2937"
)

title_label.pack(pady=20)

subtitle_label = tk.Label(
    root,
    text="AI-Powered Checkout Risk Prediction Engine",
    font=("Arial", 12),
    bg="#F4F6F9",
    fg="gray"
)

subtitle_label.pack()
```

In [41]:
```
# ========================================
# MAIN FRAME
# ========================================

main_frame = tk.Frame(
    root,
    bg="white",
    bd=2,
    relief="solid"
)

main_frame.pack(
    padx=20,
    pady=20,
    fill="both",
    expand=True
)
```

In [42]:
```
# ========================================
# INPUT FRAME
# ========================================

input_frame = tk.LabelFrame(
    main_frame,
    text="Customer Session Inputs",
    font=("Arial", 12, "bold"),
    padx=15,
    pady=15,
    bg="white"
)

input_frame.pack(
    side="left",
    fill="both",
    expand=True,
    padx=15,
    pady=15
)
```

In [43]:
```
# ========================================
# RESULT FRAME
# ========================================

result_frame = tk.LabelFrame(
    main_frame,
    text="Prediction Results",
    font=("Arial", 12, "bold"),
    padx=15,
    pady=15,
    bg="white"
)

result_frame.pack(
    side="right",
    fill="both",
    expand=True,
    padx=15,
    pady=15
)
```

In [44]:
```
# ========================================
# DROPDOWN OPTIONS
# ========================================

regions = [
    "Africa",
    "Asia",
    "Europe",
    "Middle East",
    "North America",
    "South America"
]

traffic_channels = [
    "Affiliate",
    "Direct",
    "Email",
    "Organic Search",
    "Paid Search",
    "Referral",
    "Social Media"
]

device_types = [
    "Desktop",
    "Mobile",
    "Tablet"
]

customer_segments = [
    "Discount Shopper",
    "High Value",
    "New",
    "Returning"
]

product_categories = [
    "Beauty",
    "Electronics",
    "Fashion",
    "Groceries",
    "Home & Living"
]

payment_methods = [
    "Credit Card",
    "Debit Card",
    "Digital Wallet",
    "Cash on Delivery",
    "Unknown"
]

conversion_stages = [
    "Product Viewed",
    "Added To Cart",
    "Checkout Started",
    "Purchased"
```

```
]

print("Dropdown Options Created")
```

```
Dropdown Options Created
```

In [45]:
```
# INPUT VARIABLES
# ========================================

region_var = tk.StringVar()
traffic_var = tk.StringVar()
device_var = tk.StringVar()
segment_var = tk.StringVar()
category_var = tk.StringVar()
payment_var = tk.StringVar()
stage_var = tk.StringVar()

pages_var = tk.StringVar()
duration_var = tk.StringVar()
product_view_var = tk.StringVar()
cart_var = tk.StringVar()
checkout_var = tk.StringVar()
quantity_var = tk.StringVar()
price_var = tk.StringVar()
discount_var = tk.StringVar()
discount_amount_var = tk.StringVar()
shipping_var = tk.StringVar()
marketing_var = tk.StringVar()
inventory_var = tk.StringVar()

print("Input Variables Ready")
```

```
Input Variables Ready
```

Customer Input Form

This section builds the interactive customer session input form used by the machine learning model.

Users can enter customer behavioral and session variables to predict cart abandonment risk.

In [46]:
```
# ========================================
# INPUT FORM FIELDS
# ========================================

label_font = ("Arial", 10, "bold")

# Row 0
tk.Label(
    input_frame,
    text="Region",
    font=label_font,
    bg="white"
).grid(row=0, column=0, sticky="w", pady=5)

region_dropdown = ttk.Combobox(
    input_frame,
    textvariable=region_var,
    values=regions,
    width=15
)
region_dropdown.grid(row=0, column=1, pady=5)
region_dropdown.current(0)


tk.Label(
    input_frame,
    text="Traffic Channel",
    font=label_font,
    bg="white"
).grid(row=1, column=0, sticky="w", pady=5)

traffic_dropdown = ttk.Combobox(
    input_frame,
    textvariable=traffic_var,
    values=traffic_channels,
    width=15
)
traffic_dropdown.grid(row=1, column=1, pady=5)
traffic_dropdown.current(0)


tk.Label(
    input_frame,
    text="Device Type",
    font=label_font,
    bg="white"
).grid(row=2, column=0, sticky="w", pady=5)

device_dropdown = ttk.Combobox(
    input_frame,
    textvariable=device_var,
    values=device_types,
    width=15
)
device_dropdown.grid(row=2, column=1, pady=4)
device_dropdown.current(0)
```

```
tk.Label(
    input_frame,
    text="Customer Segment",
    font=label_font,
    bg="white"
).grid(row=3, column=0, sticky="w", pady=4)

segment_dropdown = ttk.Combobox(
    input_frame,
    textvariable=segment_var,
    values=customer_segments,
    width=15
)
segment_dropdown.grid(row=3, column=1, pady=5)
segment_dropdown.current(0)


tk.Label(
    input_frame,
    text="Product Category",
    font=label_font,
    bg="white"
).grid(row=3, column=0, sticky="w", pady=4)

category_dropdown = ttk.Combobox(
    input_frame,
    textvariable=category_var,
    values=product_categories,
    width=15
)
category_dropdown.grid(row=4, column=1, pady=5)
category_dropdown.current(0)


tk.Label(
    input_frame,
    text="Payment Method",
    font=label_font,
    bg="white"
).grid(row=5, column=0, sticky="w", pady=5)

payment_dropdown = ttk.Combobox(
    input_frame,
    textvariable=payment_var,
    values=payment_methods,
    width=15
)
payment_dropdown.grid(row=5, column=1, pady=5)
payment_dropdown.current(0)
```

Out[46]: ''

In [47]:
```
# ========================================
# NUMERIC INPUT FIELDS
# ========================================

fields = [

    ("Pages Viewed", pages_var),
    ("Session Duration", duration_var),
    ("Product Viewed Flag", product_view_var),
    ("Added To Cart Flag", cart_var),
    ("Checkout Started Flag", checkout_var),
    ("Quantity", quantity_var),
    ("Product Price", price_var),
    ("Discount Percent", discount_var),
    ("Discount Amount", discount_amount_var),
    ("Shipping Cost", shipping_var),
    ("Marketing Cost", marketing_var),
    ("Inventory Stock Level", inventory_var)

]

start_row = 6

for idx, (label, variable) in enumerate(fields):

    tk.Label(
        input_frame,
        text=label,
        font=label_font,
        bg="white"
    ).grid(
        row=start_row + idx,
        column=0,
        sticky="w",
        pady=5
    )

    tk.Entry(
        input_frame,
        textvariable=variable,
        width=15
    ).grid(
        row=start_row + idx,
        column=1,
        pady=5
    )
```

In [48]:
```
# ========================================
# CONVERSION STAGE
# ========================================

tk.Label(
    input_frame,
    text="Conversion Stage",
    font=label_font,
    bg="white"
).grid(row=19, column=0, sticky="w", pady=5)

stage_dropdown = ttk.Combobox(
    input_frame,
    textvariable=stage_var,
    values=conversion_stages,
    width=25
)

stage_dropdown.grid(
    row=19,
    column=1,
    pady=5
)

stage_dropdown.current(0)
```

Out[48]: ''

Result Dashboard

This section creates the prediction result panel used to display:

- Cart abandonment probability
- Risk category
- Executive recommendation

In [49]:
```
# ========================================
# RESULT LABELS
# ========================================

prediction_label = tk.Label(
    result_frame,
    text="Cart Risk Probability: --",
    font=("Arial", 16, "bold"),
    bg="white",
    fg="#1F2937"
)

prediction_label.pack(pady=20)


risk_label = tk.Label(
    result_frame,
    text="Risk Level: --",
    font=("Arial", 16, "bold"),
    bg="white"
)

risk_label.pack(pady=10)


recommendation_label = tk.Label(
    result_frame,
    text="Executive Recommendation Appears Here",
    font=("Arial", 12),
    bg="white",
    wraplength=350,
    justify="left"
)

recommendation_label.pack(pady=15)
```

In [50]:
```
# ========================================
# PREDICTION FUNCTION
# ========================================

def predict_cart_risk():

    try:

        input_data = pd.DataFrame([{

            "Region": region_var.get(),
            "TrafficChannel": traffic_var.get(),
            "DeviceType": device_var.get(),
            "CustomerSegment": segment_var.get(),
            "ProductCategory": category_var.get(),
            "PaymentMethod": payment_var.get(),

            "PagesViewed": float(
                pages_var.get()
            ),

            "SessionDurationSeconds": float(
                duration_var.get()
            ),

            "ProductViewedFlag": int(
                product_view_var.get()
            ),

            "AddedToCartFlag": int(
                cart_var.get()
            ),

            "CheckoutStartedFlag": int(
                checkout_var.get()
            ),

            "Quantity": float(
                quantity_var.get()
            ),

            "ProductPrice": float(
                price_var.get()
            ),

            "DiscountPercent": float(
                discount_var.get()
            ),

            "DiscountAmount": float(
                discount_amount_var.get()
            ),

            "ShippingCost": float(
                shipping_var.get()
            ),
```

```
            "MarketingCost": float(
                marketing_var.get()
            ),

            "InventoryStockLevel": float(
                inventory_var.get()
            ),

            "SessionHour": 12,
            "SessionDay": "Monday",
            "SessionMonth": "January",
            "IsWeekend": 0,

            "ConversionStage":
            stage_var.get()

        }])

        probability = (
            cart_model.predict_proba(
                input_data
            )[0][1]
        )

        probability_pct = (
            probability * 100
        )

        prediction_label.config(
            text=f"Cart Risk Probability: {probability_pct:.2f}%"
        )

        # Risk Classification
        if probability_pct >= 70:

            risk_level = "HIGH RISK"

            recommendation = """
• Customer likely to abandon cart

Recommended Actions:

• Trigger abandoned cart email
• Offer 10% discount incentive
• Offer free shipping
• Simplify checkout
"""

            risk_color = "red"

        elif probability_pct >= 40:

            risk_level = "MEDIUM RISK"

            recommendation = """
• Moderate abandonment probability
```

```
Recommended Actions:

• Send reminder notification
• Recommend related products
"""

            risk_color = "orange"

        else:

            risk_level = "LOW RISK"

            recommendation = """
• Customer likely to convert

Recommended Actions:

• Recommend premium upsells
• Maintain checkout experience
"""

            risk_color = "green"

        risk_label.config(
            text=f"Risk Level: {risk_level}",
            fg=risk_color
        )

        recommendation_label.config(
            text=recommendation
        )

    except Exception as e:

        messagebox.showerror(
            "Prediction Error",
            str(e)
        )
```

In [51]:
```
# ========================================
# PREDICT BUTTON
# ========================================

predict_button = tk.Button(
    input_frame,
    text="Predict Cart Risk",
    command=predict_cart_risk,
    font=("Arial", 12, "bold"),
    bg="#2563EB",
    fg="white",
    padx=15,
    pady=10
)

predict_button.grid(
    row=21,
    column=0,
    columnspan=2,
    pady=20
)
```

In [ ]:
```

```

In [52]:
```
# ========================================
# RUN APPLICATION
# ========================================

root.mainloop()
```

In [ ]:
```

```